

# PROGRAMMING LANGUAGES

## INTEGER EXPRESSION INTERPRETER IN ANTLR

Michele Pasqua, PhD



UNIVERSITÀ  
di **VERONA**  
Dipartimento  
di **INFORMATICA**

A.Y. 2025/2026



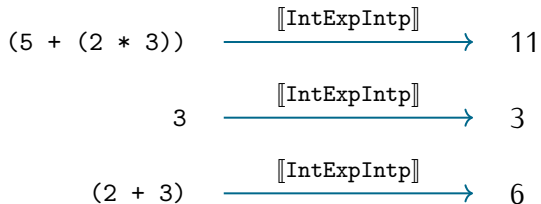
© Michele Pasqua, 2026

# FIRST INTERPRETER



# INTEGER EXPRESSION INTERPRETER

Interpreter IntExpIntp for the language of integer expressions



# PARSING AMBIGUITY

A grammar is **ambiguous** if a string can have more than one parse tree

`exp : NAT | exp ADD exp | exp MUL exp ;`

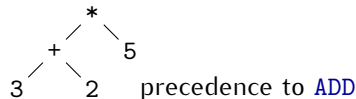
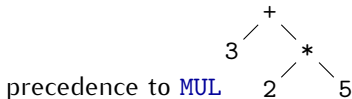
`ADD : '+' ;`

`MUL : '*' ;`

`NAT : '0' | [1-9][0-9]* ;`

How to parse 3+2\*5 ?

*// ANTLR: first-matching rule, from left to right*



# INTEGER EXPRESSION LANGUAGE GRAMMAR

Formal CF grammar, with  $n \in \mathbb{N}$ :

$$\text{exp} \rightarrow n \mid (\text{exp} + \text{exp}) \mid (\text{exp} * \text{exp})$$

ANTLR grammar:

```
exp  :  NAT                                # nat
      |  LPAR exp ADD exp RPAR            # add
      |  LPAR exp MUL exp RPAR            # mul
      ;
```

```
LPAR  :  '(' ;
```

```
RPAR  :  ')' ;
```

```
ADD   :  '+' ;
```

```
MUL   :  '*' ;
```

```
NAT   :  '0' | [1-9] [0-9]* ;
```

```
WS    :  [ \t\r\n]+ -> skip ;
```

# ANTLR REPRESENTATION

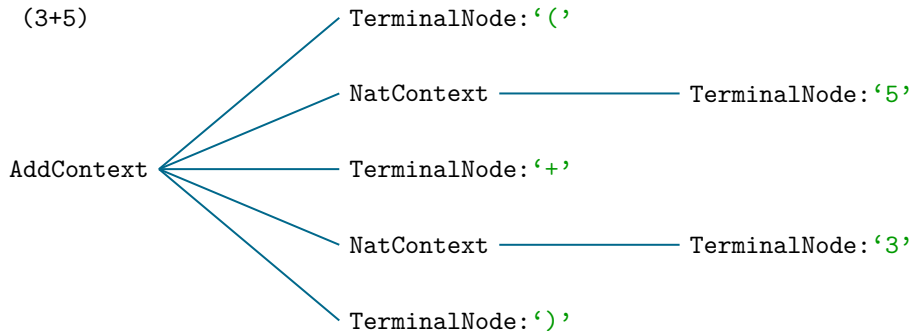
Each terminal is represented by an object `TerminalNode` carrying its string value

Each non-terminal is a **recursively visitable** object `Context`

- ▶ Created for each grammar rule
- ▶ Created for each rule label (no context for the rule is created)

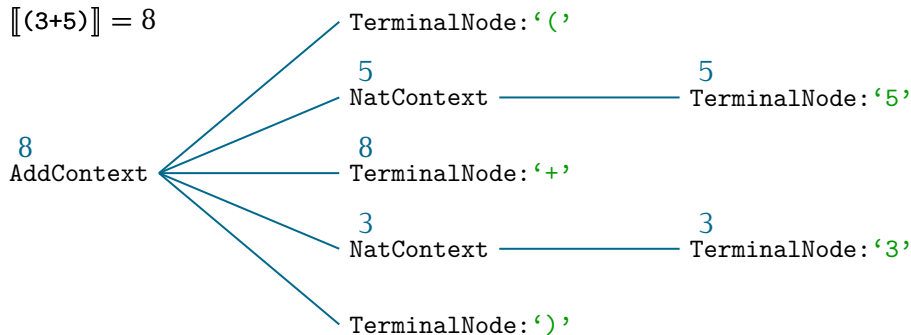
👉 Think of a `Context` as a **subtree** in the parse tree

## ANTLR REPRESENTATION (CONT'D)



# EVALUATION

We can *evaluate* an expression by recursively interpreting the parse tree



## VISITOR PATTERN

ANTLR automatically generates a `IntExpBaseVisitor` class (parse tree visitor)

- ▶ A `visit` method is defined for each `Context`
- ▶ Override these methods to compute on the parse tree

```
visit(ptree)                                // on the string (5+3)
    → visitAdd(AddContext ctxadd)
    → visit(ctxadd.exp(0)) + visit(ctxadd.exp(1))
    → visitNat(NatContext ctxnat3) + visitNat(NatContext ctxnat5)
    → 5 + 3
    → 8
```

# INTEGER EXPRESSION LANGUAGE SEMANTICS

Language **small-step** semantics

$$\text{NAT} \frac{}{n \longrightarrow n}$$

$$\text{ADDL} \frac{\text{exp}_1 \longrightarrow \text{exp}'_1}{(\text{exp}_1 + \text{exp}_2) \longrightarrow (\text{exp}'_1 + \text{exp}_2)}$$

$$\text{ADDR} \frac{\text{exp}_2 \longrightarrow \text{exp}'_2}{(n_1 + \text{exp}_2) \longrightarrow (n_1 + \text{exp}'_2)}$$

$$\text{ADD} \frac{}{(n_1 + n_2) \longrightarrow n_3} \quad n_3 = n_1 + n_2$$

Rules for multiplication are analogous

## ALTERNATIVE SEMANTICS

Define an alternative “interpreter” for the language

- ▶ Numbers evaluate to their string representation, not to their numeric value
- ▶ Operators are translated to their mnemonic representation

For instance, the input string `"(2 * (5 + 3))"` should evaluate to the **string**

`"(2 times (5 plus 3))"`

✂ Implement an interpreter with the new semantics in ANTLR

- ▶ Define the alternative small-step semantics

# ADDING FEATURES



## EXTEND THE INTEGER EXPRESSION LANGUAGE

Add support for negative integers (-1, -2, ...) and the operators

- ▶ **SUB** denoted by -, e.g.,  $(-7 - -3)$  *// subtraction*
- ▶ **DIV** denoted by /, e.g.,  $-9 / 3$  *// integer division*
- ▶ **MOD** denoted by mod, e.g.,  $-7 \text{ mod } -2$  *// division remainder*

✕ Implement an interpreter for the new language in ANTLR

## EXTEND THE INTEGER EXPRESSION LANGUAGE (CONT'D)

Add support for variables holding numbers:  $(a * (xy + 5))$

- ▶ Variable names can only be lowercase strings over the Latin alphabet
- ▶ Variables can be initialized with expressions, e.g.,  $c = ((5 * a) + 5)$
- ▶ Variables not initialized will take the value **zero**

A program is a finite sequence of variable initializations of the form

$$x_1 = \text{exp}_1; \dots x_n = \text{exp}_n;$$

followed by a **single** integer expression

## EXTEND THE INTEGER EXPRESSION LANGUAGE (CONT'D)

For instance, the following is a valid program

```
base = 5;  
height = (base + 3);  
  
(base * height)
```

and its evaluation yields the number 40

✂ Implement an interpreter for the new language in ANTLR

## SEMANTICS WITH VARIABLES

Variables require a notion of **program state**

- ▶ For instance, a map  $\sigma \in \mathbb{M} \rightarrow \mathbb{V}$  from variables to values
- ▶ Where  $\sigma(x) = 5$  means that the variable  $x$  holds the value 5

To **what** variable initializations evaluate?

- ▶ They transform program states, e.g., changing the value of a variable

$$\text{VARINIT} \frac{\langle \sigma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \sigma, x = \text{exp} \rangle \longrightarrow \langle \sigma, x = \text{exp}' \rangle}$$

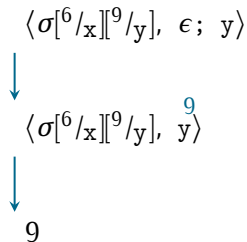
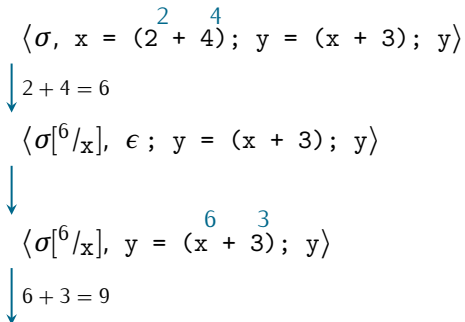
$$\text{VARINITE} \frac{-}{\langle \sigma, x = n \rangle \longrightarrow \langle \sigma[n/x], \epsilon \rangle}$$

$$\text{VAR} \frac{-}{\langle \sigma, x \rangle \longrightarrow \sigma(x)}$$

$$\text{NAT} \frac{-}{\langle \sigma, n \rangle \longrightarrow n}$$

## SEMANTICS WITH VARIABLES (CONT'D)

How is  $x = (2 + 4); y = (x + 3); y$  evaluated ?



Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

**You are free to:**

 **Share** – Copy, distribute, and display the work

**Under the following terms:**

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.